

Benutzersuche in DB

Do., 18. Juni um 14:54

wie kann man einen user in der db suchen zur anmeldung in python vs code

Wenn du Python mit einer Datenbank (z. B. SQLite) verwendest, sucht man bei der Anmeldung normalerweise den Benutzer anhand seiner E-Mail oder seines Benutzernamens und vergleicht dann das Passwort (als Hash).

Mit Flask-SQLAlchemy

Angenommen, dein `User`-Modell sieht so aus:

Python

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(100), unique=True, nullable=False)
    password_hash = db.Column(db.String(200), nullable=False)
```

Dann kannst du bei der Anmeldung den Benutzer so suchen:

Python

```
from werkzeug.security import check_password_hash

email = request.form.get("email")
password = request.form.get("password")

# Benutzer anhand der E-Mail suchen
user = User.query.filter_by(email=email).first()

# Prüfen, ob Benutzer existiert und Passwort stimmt
if user and check_password_hash(user.password_hash, password):
    login_user(user)
    return redirect(url_for("main.index"))
else:
    return render_template("login.html", error="Ungültige E-Mail oder Passwort")
```

Mit SQLite ohne SQLAlchemy

Python

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

email = input("E-Mail: ")

cursor.execute(
    "SELECT * FROM users WHERE email = ?",
    (email,)
)

user = cursor.fetchone()

if user:
    print("Benutzer gefunden")
else:
```

```
print("Benutzer nicht gefunden")  
conn.close()
```

Erklärung

- `SELECT * FROM users WHERE email = ?` sucht nach einem Benutzer mit der angegebenen E-Mail.
- `fetchone()` liefert den ersten gefundenen Datensatz oder `None`, falls keiner existiert.
- Das Passwort sollte **nie im Klartext** gespeichert werden, sondern als Hash (z. B. mit `generate_password_hash()` und `check_password_hash()`).

Falls du Flask, SQLAlchemy oder SQLite verwendest, kann ich dir ein komplettes Login-Beispiel mit Registrierung und Datenbank zeigen.